Department of CSE, PES University

UE24CS341A: SOFTWARE ENGINEERING

5TH SEMESTER, ACADEMIC YEAR 2026-27

LAB 7: FUZZING FOR INPUT VALIDATION BUGS DATE: 11-11-2026

NAME: PRATHAM KUMAR

SRN NO.: PES2UG24CS368

PRN NO.: PES2202400086

ASSIGNMENT WORK

Objective

To explore fuzz testing as a security and quality assurance technique by identifying and fixing bugs in the python script provided to you using the Hypothesis fuzzing library.

Duration: 60 minutes

Software Requirements

- Python 3.8+

Learning Outcomes

By the end of this lab, students will:

- Understand the purpose and methodology of fuzz testing.
- Apply property-based fuzzing using Hypothesis.
- Detect and analyse bugs caused by edge-case inputs.
- Improve code robustness through defensive programming.

Deliverables:

1. PDF Document with the screenshots
2. Fixed application python script
3. Updated test_processor.py code

Department of CSE, PES University

UE24CS341A: SOFTWARE ENGINEERING
5TH SEMESTER, ACADEMIC YEAR 2026-27
LAB 7: FUZZING FOR INPUT VALIDATION BUGS DATE: 11-11-2026

1. INTRODUCTION

Fuzzing, also known as Fuzz Testing, is a dynamic software testing technique used to identify errors, unexpected behaviour, and potential vulnerabilities in an application by providing it with a wide variety of random, unexpected, malformed, or invalid inputs. The primary objective of fuzz testing is to evaluate how reliably an application handles inputs that may not have been anticipated during normal development and testing.

Traditional testing generally relies on predefined test cases written to verify expected functionality. However, such test cases may not cover every possible input combination or unusual edge case. Fuzz testing addresses this limitation by automatically generating numerous inputs and observing how the application responds. This helps identify issues such as runtime exceptions, incorrect behaviour, and application crashes that may otherwise remain undetected.

In this experiment, Hypothesis, a property-based testing library for Python, is used to perform automated fuzz testing. Hypothesis allows developers to define properties that a function is expected to satisfy and automatically generates different input values to verify those properties. It also identifies and simplifies failing inputs, making it easier to understand and reproduce the underlying problems.

The experiment focuses on the faculty-provided Marathon Registration Parser Python application, which consists of three functions: format_bib_name(), parse_finish_time(), and average_split(). These functions are responsible for formatting a runner's name, converting a marathon finish time from H:MM:SS format into seconds, and calculating the average of split times, respectively. The original implementation assumes that the supplied inputs are valid and properly formatted, which may result in runtime errors when unexpected or invalid inputs are provided.

The primary objective of this experiment is to identify and rectify these input-validation issues using Hypothesis-based fuzz testing. Initially, the original application is executed to understand its normal behaviour. Hypothesis test cases are then implemented and executed to automatically generate different inputs and identify potential failures. Based on the errors discovered, the application is modified using appropriate input validation and defensive programming techniques. Finally, the corrected application is executed again, and the same Hypothesis test suite is rerun to verify that the identified issues have been addressed.

The procedure followed to accomplish the objectives of this experiment is described in the following steps.

2. PROCEDURE

Step 1: Installing Dependencies

The first step of the experiment involves installing the required Python libraries for performing fuzz testing and executing automated test cases. Two dependencies, Hypothesis and pytest, are required to complete this experiment.

Department of CSE, PES University

UE24CS341A: SOFTWARE ENGINEERING

5TH SEMESTER, ACADEMIC YEAR 2026-27

LAB 7: FUZZING FOR INPUT VALIDATION BUGS **DATE: 11-11-2026**

1. Hypothesis

Hypothesis is a property-based testing library for Python that automatically generates a wide variety of input values to test the behaviour of functions. It helps identify unexpected inputs, edge cases, and potential runtime errors that may not be covered by manually written test cases.

2. pytest

pytest is a Python testing framework used to execute test functions and evaluate their results. It provides a convenient way to run the Hypothesis-based test cases, identify failures, and verify whether the application behaves as expected after modifications.

Installation Commands

The following commands were executed in the PowerShell terminal from the LAB7 project directory:

- python -m pip install hypothesis
- python -m pip install pytest

To verify the installation, the following commands were executed:

- python -m pytest --version
- python -m pip show hypothesis

The installation and verification confirmed that both dependencies were available in the Python environment.

```
PS C:\Users\HP\Documents\L6_L7_60_marathon_registration\LAB7> python -m pip install pytest
hypothesis
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pytest in C:\Users\HP\AppData\Roaming\Python\Python314\site-
packages (9.2.0)
Collecting hypothesis
  Downloading hypothesis-6.170.15-cp314-cp314-win_amd64.whl.metadata (5.6 kB)
Requirement already satisfied: colorama>=0.4 in C:\Users\HP\AppData\Roaming\Python\Python31
4\site-packages (from pytest) (0.4.6)
Requirement already satisfied: iniconfig>=2 in C:\Users\HP\AppData\Roaming\Python\Python314
\site-packages (from pytest) (2.3.1)
Requirement already satisfied: packaging>=24 in C:\Users\HP\AppData\Roaming\Python\Python31
4\site-packages (from pytest) (26.0)
Requirement already satisfied: pluggy<2,>=1.6 in C:\Users\HP\AppData\Roaming\Python\Python3
14\site-packages (from pytest) (1.7.0)
Requirement already satisfied: pygments>=2.15 in C:\Users\HP\AppData\Roaming\Python\Python3
14\site-packages (from pytest) (2.21.0)
Collecting sortedcontainers<3.0.0,>=2.1.0 (from hypothesis)
  Downloading sortedcontainers-2.4.0-py2.py3-none-any.whl.metadata (10 kB)
Downloading hypothesis-6.170.15-cp314-cp314-win_amd64.whl (691 kB)
   ---------------------------------------- 691.1/691.1 kB 7.7 MB/s  0:00:00
Downloading sortedcontainers-2.4.0-py2.py3-none-any.whl (29 kB)
Installing collected packages: sortedcontainers, hypothesis
   ----------                               1/2 [hypothesis]  WARNING: The script hypothesi
s.exe is installed in 'C:\Users\HP\AppData\Roaming\Python\Python314\Scripts' which is not o
n PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --
no-warn-script-location.
Successfully installed hypothesis-6.170.15 sortedcontainers-2.4.0
PS C:\Users\HP\Documents\L6_L7_60_marathon_registration\LAB7> python -m pytest --version
pytest 9.2.0
PS C:\Users\HP\Documents\L6_L7_60_marathon_registration\LAB7> python -m pip show hypothesis

Name: hypothesis
Version: 6.170.15
Summary: The property-based testing library for Python
Home-page: https://hypothesis.works
Author:
Author-email: "David R. MacIver and Zac Hatfield-Dodds" <david@drmaciver.com>
License-Expression: MPL-2.0
Location: C:\Users\HP\AppData\Roaming\Python\Python314\site-packages
Requires: sortedcontainers
Required-by:
PS C:\Users\HP\Documents\L6_L7_60_marathon_registration\LAB7>
```

Figure 1: Installation and Verification of Hypothesis and pytest Dependencies.

Explanation: The screenshot demonstrates the successful installation and verification of the required Python testing libraries. Hypothesis version 6.170.15 and pytest version 9.2.0 were confirmed in the development environment, establishing the prerequisites for implementing and executing automated fuzz tests.

Outcome of Step 1: Both required dependencies, Hypothesis and pytest, were successfully installed and verified. The Python environment was therefore ready for executing the original application and performing property-based fuzz testing.

Step 2: Running the Original Buggy Application
After successfully installing the required dependencies, the next step involves executing the original Python application provided in the assigned GitHub repository under the LAB7 directory.

The application used for this experiment is marathon_registration_parser.py, which processes marathon registration details through three functions:

1. format_bib_name() – Converts the runner's name into uppercase and limits it to 12 characters for printing on the race bib.
2. parse_finish_time() – Converts a marathon finish time provided in H:MM:SS format into its equivalent value in seconds.
3. average_split() – Calculates the average of the runner's split times provided as space-separated values in minutes.

Command Executed
The following command was executed in the PowerShell terminal from the LAB7 directory to run the original application: python marathon_registration_parser.py

For verifying the normal execution of the application, the following sample inputs were provided:

Function	Input
format_bib_name()	Pratham Kumar
parse_finish_time()	3:45:10
average_split()	25 26 24

Observed Output
The application successfully processed the provided valid inputs and generated the following results:

- Bib name: PRATHAM KUMA
- Finish time in seconds: 13510
- Average split (minutes): 25.0

These results demonstrate that the original application performs its intended operations correctly when supplied with properly formatted inputs. However, this execution only verifies normal input behaviour and does not

establish whether the application can handle unexpected, malformed, or invalid inputs without crashing. Therefore, fuzz testing is required to identify potential input-validation issues.

```
PS C:\Users\HP\Documents\L6_L7_60_marathon_registration\LAB7> python marathon_registration_
parser.py
==== Running format_bib_name ====
Enter the name to print on the bib: Pratham Kumar
Bib name: PRATHAM KUMA

==== Running parse_finish_time ====
Enter a finish time as H:MM:SS (e.g. 3:45:10): 3:45:10
Finish time in seconds: 13510

==== Running average_split ====
Enter your 5 km split times in minutes separated by spaces (e.g. 25 26 24):25 26 24
Average split (minutes): 25.0
PS C:\Users\HP\Documents\L6_L7_60_marathon_registration\LAB7>
```

Figure 2: Execution of the Original Marathon Registration Parser Application.
Explanation: The screenshot demonstrates the successful execution of the original application using valid sample inputs. The runner's name is converted to uppercase and truncated to 12 characters, the finish time is converted into seconds, and the average split time is calculated correctly. This execution establishes the baseline behaviour of the application before performing fuzz testing.

Outcome of Step 2: The original Marathon Registration Parser application was successfully executed, and all three functions produced the expected outputs for valid inputs. The application was now ready for property-based fuzz testing to identify potential failures caused by unexpected or invalid inputs.

Step 3: Completing the Hypothesis Test File

After executing the original Marathon Registration Parser application, the next step involves completing the test file using the Hypothesis property-based testing library. The objective is to automatically generate a wide variety of inputs and identify potential edge cases that may cause the application's functions to fail.
For this purpose, the faculty-provided test template, test_marathon_registration_parser.py, was used. The template contained three incomplete test functions corresponding to the three functions implemented in the original application.

1. Importing the Required Modules
The following modules were imported into the test file:

```
from hypothesis import given, strategies as st
from marathon_registration_parser import (
    format_bib_name,
    parse_finish_time,
    average_split,
)
```

- given: A Hypothesis decorator that allows test functions to receive automatically generated inputs.
- strategies as st: Provides strategies for generating different types of test inputs.
- The three parser functions are imported from the application module so that they can be tested directly.

2. Defining the Hypothesis Test Functions
Three test functions were completed, one for each parser function.

The strategy st.text() | st.none() was used to generate arbitrary Unicode strings as well as None values. This allows the application to be tested with both ordinary and unexpected inputs.

The following test functions were implemented:

a. Testing format_bib_name()

```
@given(st.text() | st.none())
def test_format_bib_name_no_crash(s):
    try:
        format_bib_name(s)
    except Exception as error:
        assert False, (
            f"format_bib_name crashed for input {s!r}: {error}"
        )
```

This test checks whether the name-formatting function can handle generated strings and None values without raising an exception.

b. Testing parse_finish_time()

```
@given(st.text() | st.none())
def test_parse_finish_time_no_crash(s):
    try:
        parse_finish_time(s)
    except Exception as error:
        assert False, (
            f"parse_finish_time crashed for input {s!r}: {error}"
        )
```

This test verifies whether the finish-time parsing function can safely handle arbitrary strings and None values, including incorrectly formatted time inputs.

c. Testing average_split()

```
@given(st.text() | st.none())
def test_average_split_no_crash(s):
    try:
```

```
        average_split(s)
    except Exception as error:
        assert False, (
            f"average_split crashed for input {s!r}: {error}"
        )
```

This test checks whether the split-average function can handle arbitrary strings and None values without encountering errors such as invalid numeric conversion or division by zero.

3. Understanding the Testing Approach

The three test functions follow the same testing approach:

- Hypothesis generates different input values using st.text() | st.none().
- Each generated input is passed to its corresponding application function.
- The try block allows the function to execute.
- If an exception occurs, the except block converts it into a test failure and reports the input that caused the failure.

This approach verifies the no-crash property, ensuring that the functions do not terminate unexpectedly when supplied with generated inputs.

Completed Test File

All three test functions were completed in the faculty-provided test_marathon_registration_parser.py file. The completed implementation was saved in the LAB7 directory alongside the application module.

```
PS C:\Users\HP\Documents\L6_L7_60_marathon_registration\LAB7> Get-Content test_marathon_registration_parser.py

from hypothesis import given, strategies as st

from marathon_registration_parser import (
    format_bib_name,
    parse_finish_time,
    average_split,
)


@given(st.text() | st.none())
def test_format_bib_name_no_crash(s):
    try:
        format_bib_name(s)
    except Exception as error:
        assert False, (
            f"format_bib_name crashed for input {s!r}: {error}"
        )


@given(st.text() | st.none())
def test_parse_finish_time_no_crash(s):
    try:
        parse_finish_time(s)
    except Exception as error:
        assert False, (
            f"parse_finish_time crashed for input {s!r}: {error}"
        )


@given(st.text() | st.none())
def test_average_split_no_crash(s):
    try:
        average_split(s)
    except Exception as error:
        assert False, (
            f"average_split crashed for input {s!r}: {error}"
        )
PS C:\Users\HP\Documents\L6_L7_60_marathon_registration\LAB7> |
```

Figure 3: Implementation of Hypothesis-Based Test Cases for the Marathon Registration Parser.

Explanation: The completed test file contains three Hypothesis-based test functions corresponding to the application's name-formatting, finish-time parsing, and split-average operations. Each test uses automatically generated text and None inputs to identify unexpected runtime failures.

Outcome of Step 3: The faculty-provided Hypothesis test template was successfully completed with three property-based test functions. These tests were designed to explore unexpected inputs automatically and identify potential input-validation bugs in all three application functions. The completed test file was ready to be executed against the original application.

Step 4: Executing the Hypothesis Tests on the Original Application

Objective: To execute the completed Hypothesis test suite against the original, unmodified application and identify failures caused by unexpected or invalid inputs.

4.1 Test Execution

The completed test file is executed using the following command from the LAB7 directory:

- python -m pytest test_marathon_registration_parser.py

The python -m pytest command is used because PowerShell did not recognize pytest as a standalone command. This is an alternative execution method permitted by the lab instructions.

4.2 Analysis of Test Results

When the tests are executed against the original application, all three test functions fail. Hypothesis automatically generates different inputs, including empty strings, None, and malformed values, to identify cases in which the application raises runtime exceptions.

The following issues are identified:

Function	Generated invalid input	Identified issue
format_bib_name()	None	AttributeError because None does not support the strip() method.
parse_finish_time()	Empty string ('')	ValueError because the input does not contain three colon-separated parts.
parse_finish_time()	None	AttributeError because None does not support the split() method.
average_split()	':'	ValueError because the input cannot be converted into an integer.
average_split()	Empty string ('')	ZeroDivisionError because the function attempts to divide by zero when no split values are provided.
average_split()	None	AttributeError because None does not support the split() method.

These failures demonstrate that the original application does not adequately validate its inputs before performing string operations, numeric conversions, and arithmetic calculations.

4.3 Screenshot of Test Execution

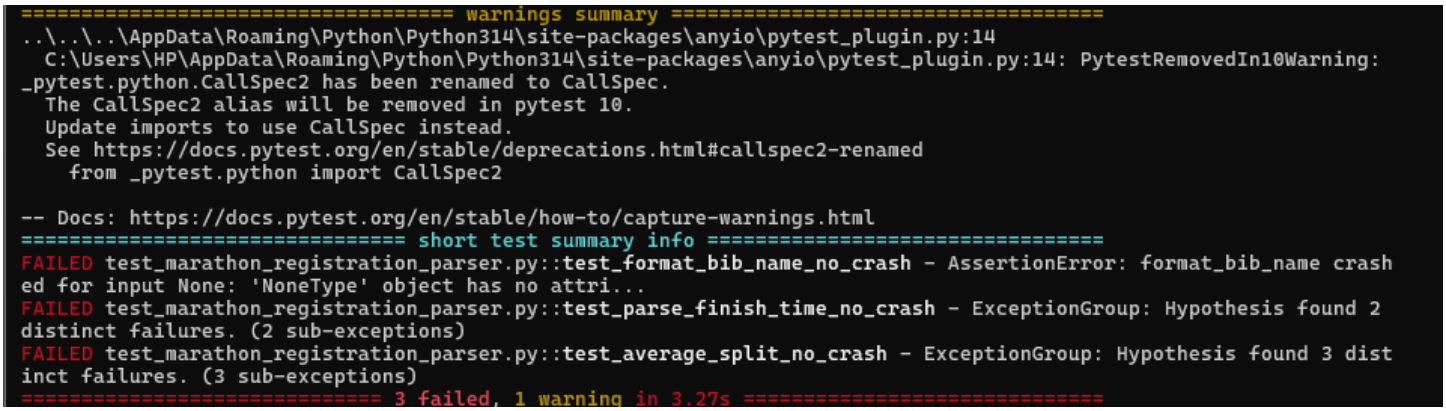


Figure 4: Execution of Hypothesis Tests Identifying Input-Validation Errors in the Original Application.

Screenshot Explanation: The screenshot shows the execution of the completed Hypothesis test suite against the original Marathon Registration Parser application. All three test functions fail because the application encounters unexpected or invalid inputs that are not handled appropriately.

4.4 Observed Outcome

The test execution produces the following summary:

Test execution summary
3 failed
1 warning · 3.27 seconds

The warning relates to a deprecation notice from the installed AnyIO pytest plugin and is not an application test failure.

The results confirm that all three functions require improved input validation and defensive programming. These identified issues will be addressed in the next step by modifying the original application.

Step 5: Fixing the Buggy Application

Objective: To rectify the input-validation errors identified during fuzz testing by modifying the original Marathon Registration Parser application and implementing appropriate defensive programming techniques.

5.1 Modifying the Application

Based on the failures identified in Step 4, the original marathon_registration_parser.py file is modified to handle unexpected, malformed, and invalid inputs safely.

The following improvements are implemented:

1. format_bib_name()

- Checks whether the input is a string.

- Removes leading and trailing whitespace using strip().
- Checks whether the input is empty.
- Returns None for invalid or empty input.
- Converts valid names to uppercase and limits them to 12 characters.

2. parse_finish_time()
- Verifies that the input is a string.
- Removes unnecessary whitespace and splits the input using the colon (:) separator.
- Ensures that exactly three components are provided.
- Checks that all components contain ASCII digits.
- Handles numeric conversion errors using exception handling.
- Validates that minutes and seconds are less than 60.
- Returns None when the input is invalid.

3. average_split()
- Verifies that the input is a string.
- Splits the input into individual values.
- Checks that at least one split value is provided.
- Ensures that all values contain ASCII digits.
- Handles numeric conversion errors.
- Returns None for invalid or empty input and calculates the average for valid values.

4. Updating the main() Function
The main function is also modified to handle invalid results gracefully. Instead of allowing invalid input to terminate the program, it displays appropriate messages such as Invalid name, Invalid finish time, and Invalid split times.

5.2 Updated Application Code
The following is the corrected implementation of marathon_registration_parser.py:

```
def format_bib_name(name):
    """
    Formats a runner's name for the race bib.
    Returns an uppercase name limited to 12 characters.
    Returns None for invalid or empty input.
    """
    if not isinstance(name, str):
        return None

    name = name.strip()
```

```
    if not name:
        return None

    return name.upper()[:12]


def parse_finish_time(text):
    """
    Converts a finish time in H:MM:SS format into seconds.
    Returns None for invalid input.
    """
    if not isinstance(text, str):
        return None

    parts = text.strip().split(':')

    if len(parts) != 3:
        return None

    if not all(part.isascii() and part.isdigit()
               for part in parts):
        return None

    try:
        hours, minutes, seconds = map(int, parts)
    except (ValueError, OverflowError):
        return None

    if minutes >= 60 or seconds >= 60:
        return None

    return hours * 3600 + minutes * 60 + seconds


def average_split(text):
    """
    Calculates the average of space-separated split times.
    Returns None for invalid or empty input.
    """
```

```
    if not isinstance(text, str):
        return None

    values = text.split()

    if not values:
        return None

    if not all(value.isascii() and value.isdigit()
               for value in values):
        return None

    try:
        splits = [int(value) for value in values]
    except (ValueError, OverflowError):
        return None

    return sum(splits) / len(splits)


def main():
    print("==== Running format_bib_name ====")
    value = input("Enter the name to print on the bib: ")
    result = format_bib_name(value)
    print("Bib name:", result if result is not None
          else "Invalid name")

    print("\n==== Running parse_finish_time ====")
    value = input(
        "Enter a finish time as H:MM:SS (e.g. 3:45:10): "
    )
    result = parse_finish_time(value)
    print("Finish time in seconds:",
          result if result is not None
          else "Invalid finish time")

    print("\n==== Running average_split ====")
    value = input(
        "Enter your 5 km split times in minutes "
        "separated by spaces (e.g. 25 26 24): "
```

```
    )
    result = average_split(value)
    print("Average split (minutes):",
          result if result is not None
          else "Invalid split times")


if __name__ == "__main__":
    main()
```

5.3 Executing the Corrected Application

After modifying and saving the application, execute it using: python marathon_registration_parser.py

Provide the following valid sample inputs:

Input field	Sample input
Runner's name	Pratham Kumar
Finish time	3:45:10
Split times	25 26 24

Expected output:

- Bib name: PRATHAM KUMA
- Finish time in seconds: 13510
- Average split (minutes): 25.0

5.4 Screenshot of the Corrected Application

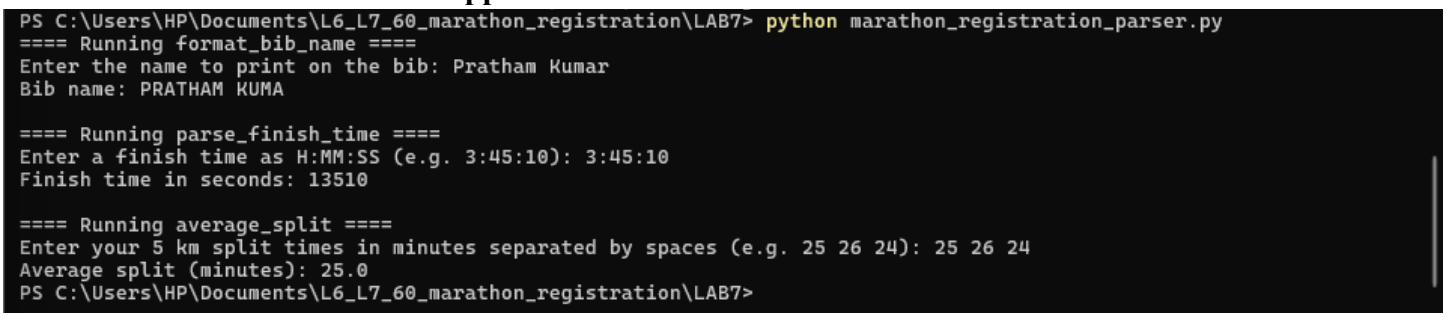


Figure 5: Execution of the Corrected Marathon Registration Parser Application.

Screenshot Explanation: The screenshot demonstrates the successful execution of the modified application using valid sample inputs. All three functions produce the expected results without runtime errors.

5.5 Observed Outcome

The original application has been modified with appropriate input validation, exception handling, and defensive programming techniques. The corrected functions now handle invalid inputs safely by returning None, while the main function displays meaningful error messages.

The corrected application executes successfully with valid inputs. The same Hypothesis test suite will now be executed again in the next step to verify whether the identified failures have been resolved.

Step 6: Re-executing the Hypothesis Tests on the Corrected Application

Objective: To verify the effectiveness of the modifications made in Step 5 by executing the same Hypothesis-based test suite against the corrected Marathon Registration Parser application.

6.1 Test Execution

After correcting the application, the same test file, test_marathon_registration_parser.py, is executed again without modifying the test cases.

Run the following command from the LAB7 directory: python -m pytest test_marathon_registration_parser.py
Using the same test suite ensures that the corrected application is evaluated against the same input-generation properties that identified the original defects.

6.2 Analysis of Test Results

The Hypothesis test suite successfully executes all three test functions:

1. test_format_bib_name_no_crash() – verifies that the name-formatting function handles generated strings and None without raising exceptions.
2. test_parse_finish_time_no_crash() – verifies that the finish-time parser safely handles valid, malformed, empty, and unexpected inputs.
3. test_average_split_no_crash() – verifies that the average-split function handles generated inputs without runtime exceptions, including empty and invalid values.

All tests passed
The final test execution produces the following result:

Final test execution summary
3 passed
1 warning · 0.48 seconds

All three Hypothesis test functions pass successfully, indicating that the identified runtime exceptions have been addressed.

6.3 Screenshot of Passing Hypothesis Tests

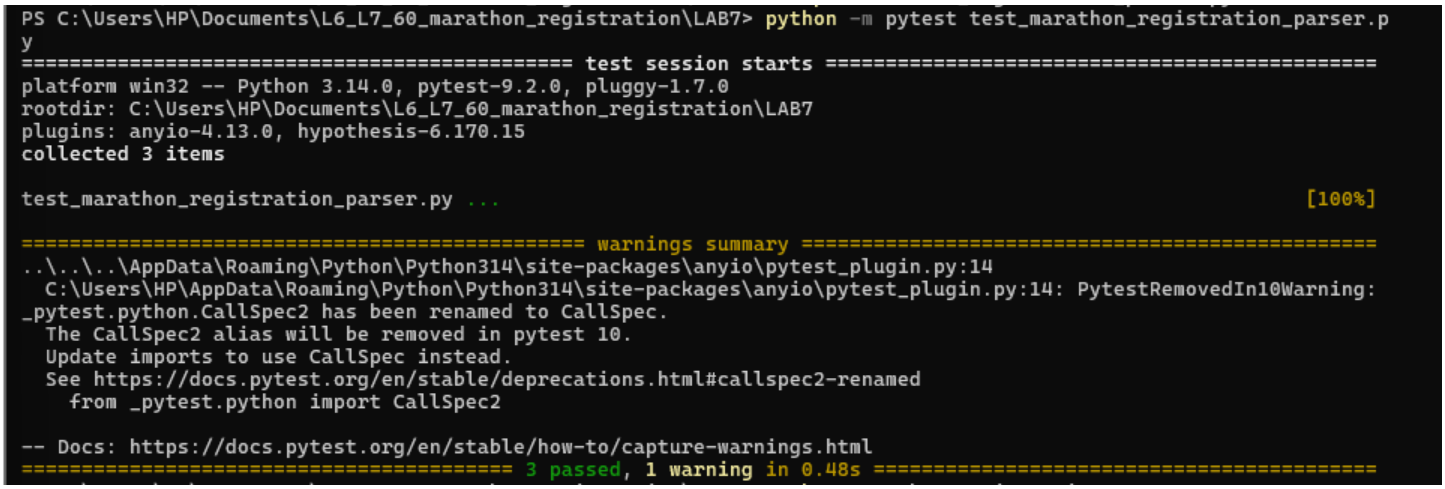


Figure 6: Successful Execution of Hypothesis Tests on the Corrected Application.
Screenshot Explanation: The screenshot demonstrates that all three Hypothesis test functions pass after the application has been modified. The result confirms that the corrected functions can handle the generated inputs without raising runtime exceptions.

6.4 Observed Outcome

The successful execution of all three test functions confirms that the input-validation issues identified in the original application have been addressed. The corrected implementation handles unexpected and invalid inputs safely through input validation, exception handling, and defensive programming.

The same Hypothesis test suite that initially identified three failing functions now reports three passing tests. This demonstrates the effectiveness of fuzz testing in discovering input-validation defects and verifying the robustness of the corrected application.

3. CONCLUSION

In this experiment, fuzz testing was performed on the Marathon Registration Parser application using the Hypothesis property-based testing library. Three functions were examined: format_bib_name(), parse_finish_time(), and average_split().

Initially, the original application failed all three Hypothesis tests because it could not safely handle certain unexpected and invalid inputs. These failures helped identify issues such as AttributeError, ValueError, and ZeroDivisionError.

The application was subsequently improved using input validation, exception handling, and defensive programming techniques. After implementing the corrections, the same Hypothesis test suite was executed again, and all three tests passed successfully.

Thus, the experiment demonstrated how fuzz testing can automatically identify edge cases, expose input-validation weaknesses, and help improve the reliability and robustness of software applications.

NAME: PRATHAM KUMAR

SRN NO.: PES2UG24CS368

PRN NO.: PES2202400086

SECTION: F

THANK YOU